NovaRent: lógica de negocio para renta de automóviles

Producto Integrador de Aprendizaje · Componentes de Lógica de Negocio

Equipo: [Cesar Maximiliano Peña Mendoza - Diego Alberto Sierra Mendoza - Arturo Sebastian Pérez Zapata] · Profesor: Miguel Ángel Gómez Marroquín · FIME-UANL · [12 de Noviembre del 2026]

El problema: reclamaciones y flota invisible

1

Disponibilidad desactualizada

Sobreventas y clientes sin unidad (N1, N4)

2

Daños sin evidencia confiable

Reclamaciones por cargos que no se pueden conciliar (N2, N8)

3

Devolución → limpieza → renta, a mano

Unidades paradas y rotación lenta (N7)

Objetivo

Digitalizar el ciclo completo de la renta con checklist y evidencia, cargos automáticos y visibilidad de flota en tiempo real.

Criterio de éxito: −30 % de reclamaciones y −20 % de tiempo de rotación.

Seis roles, cada uno con permisos propios

CLIENTE

Renta, paga y puede disputar cargos. Solo ve lo suyo.

AGENTE

Valida documentos, entrega y recibe. Extiende hasta 48 h.

PATIO

Captura el checklist con fotos y daños.

LIMPIEZA

Completa la tarea y libera la unidad.

ADMINISTRACIÓN

Autoriza upgrades, extensiones > 48 h, daños y disputas.

SISTEMA

Jobs: no-show y liberación de depósitos.

De 10 necesidades a 20 historias Gherkin

10
necesidades del negocio

20
historias de usuario (Gherkin)

17
reglas de negocio trazadas

10
ADR de arquitectura

Épicas

Reservación
US01–04

Entrega
US05–07

Excepción
US08–10

Extensión
US11–12

Devolución y cargos
US13–17

Cierre y reportes
US18–20

Reglas del negocio: parámetros, no código

Regla	Valor	Verificada con
Cancelación sin penalización	≥ 24 h antes del inicio	borde de 24 h exactas
No-show	se libera a las 2 h	1 h 59 min vs 2 h
Kilometraje	200 km/día; $3 por km extra	400 vs 401 km
Combustible faltante	cargo si falta más de 1/8	1/8 vs 2/8
Devolución tardía	29 min de tolerancia; por hora; tope 1 día	29 vs 30 min
Extensión	agente ≤ 48 h; más, Administración; ≥ 2 h antes	48 vs 49 h
Depósito de garantía	5 días hábiles, sin incidencias	jueves vs viernes + festivos

Cada regla es una Specification o una política parametrizada, con pruebas en el valor límite.

Arquitectura: el dominio no conoce frameworks

API REST · FastAPI

HTTP ⇄ comandos y DTO; mapa único de errores

Aplicación · casos de uso

Transacciones, permisos por rol, publicación de eventos

DOMINIO

Aggregates · Value Objects · Specifications · Políticas · Máquinas de estado · Eventos

Infraestructura · implementa los puertos

SQLAlchemy · PostgreSQL / SQLite · pasarela · evidencia · calendario

Puertos

- `UnidadDeTrabajo`
- `Reloj`
- `PasarelaPago`
- `AlmacenEvidencia`
- `BusEventos`
- `CalendarioHabil`

Un solo lugar —composicion.py— conoce las implementaciones.

Un aggregate por transacción, el resto por eventos

Aggregates

- `Reservacion`
- `ContratoRenta`
- `InspeccionVehiculo`
- `Liquidacion`
- `Vehiculo`
- `TareaLimpieza`

Evento → reacciones

`ContratoAbierto`	Reservación CONVERTIDA · Vehículo RENTADO · Abre liquidación y depósito
`VehiculoDevuelto`	Calcula cargos · Vehículo EN_LIMPIEZA · Crea tarea de limpieza
`TareaLimpiezaCompletada`	Vehículo DISPONIBLE
`DepositoLiberado`	Contrato CERRADO

Tres patrones que concentran las reglas

Specification

```
def cliente_apto():
  return (ClienteActivo()
        & LicenciaVigente()
        & TarjetaValida())

# fallas() explica
# 'Cliente no apto:
#  LicenciaVigente'
```

11 reglas · AND / OR / NOT

Máquina de estados

```
(CREADA, 'cancelar')
   → CANCELADA
(CREADA, 'liberar_no_show')
   → LIBERADA_NO_SHOW
(UPGRADE_OFRECIDO,
 'aceptar_upgrade')
   → UPGRADE_ACEPTADO
```

5 tablas declarativas

Eventos de dominio

1. commit del aggregate
2. se recogen sus eventos
3. el bus los publica
4. cada handler abre su propia transacción
5. y es idempotente

19 eventos · repetibles sin duplicar efectos

API REST: contrato de errores y autorización real

HTTP	Cuándo
200 / 201	Éxito / creado
400	Dato mal formado
401 · 403	Sin identidad · rol no autorizado
404	No existe
409	Transición inválida · sin disponibilidad
422	Regla de negocio o esquema de entrada

Seguridad de negocio

- Un cliente solo opera sobre lo suyo
- Cada rol, solo sus casos de uso
- Checklist firmado: inmutable
- Fotos con URI única: no se sobrescribe evidencia
- Nunca se guarda el número de tarjeta
- Un fallo deja rollback, no datos a medias

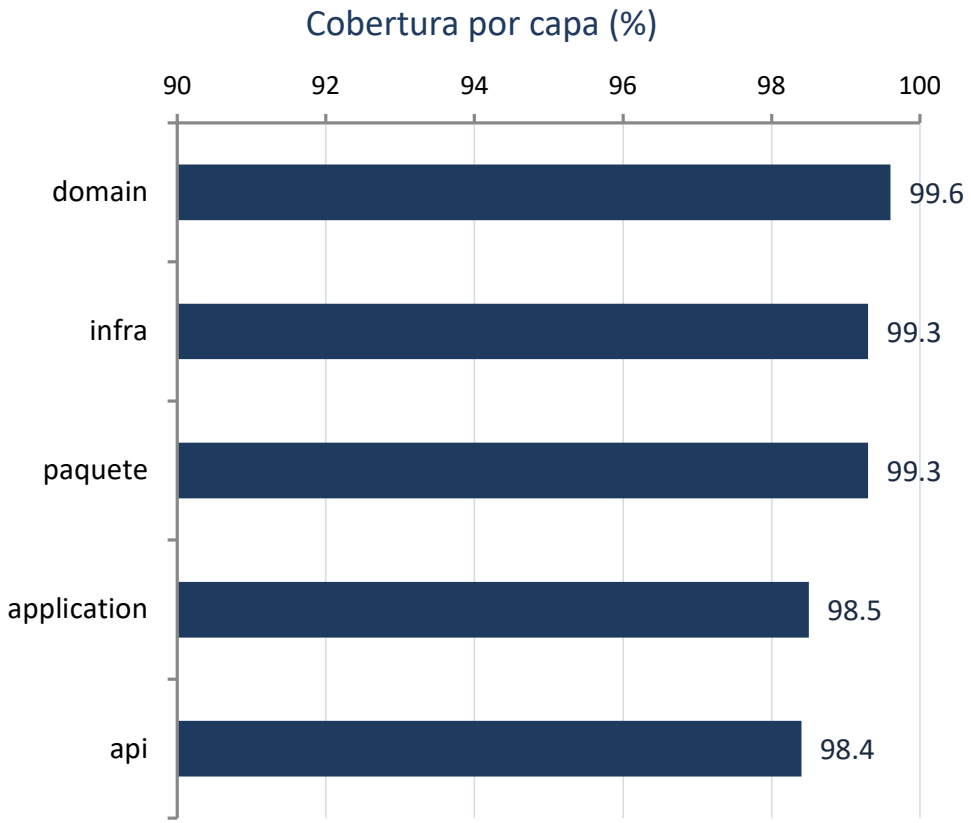
Calidad: 354 pruebas y 98.5 % de cobertura

354
pruebas (356 en PostgreSQL)

98.5 %
cobertura de líneas y ramas

Defectos reales que encontraron las pruebas

- Fotos con igual nombre se sobrescribían
- Categoría deshabilitada validada tarde
- Handlers no idempotentes tras avanzar el estado
- Swagger mostraba un solo encabezado de seguridad



Demostración: de la reservación al cierre



1	2	3	4	5	6	7
Reservar	**Entregar**	**Contrato**	**Extender**	**Devolver**	**Disputar**	**Cerrar**
garantiza categoría	checklist firmado	depósito retenido	72 h → Administración	cargos automáticos	evidencia comparada	depósito liberado

```
# 25 llamadas HTTP; termina con «Demo completa»
PERMITIR_RELOJ_SIMULADO=true docker compose up --build
python scripts/demo_flujo.py http://localhost:8000
```

Entrega: Docker Compose, CI y despliegue

Docker Compose

db · PostgreSQL 16 con healthcheck
app · FastAPI (uvicorn)
tests · pytest sobre una base separada

docker compose up --build

GitHub Actions

1. ruff check
2. pytest en Python 3.11 y 3.12 contra PostgreSQL, cobertura ≥ 80 %
3. docker compose: pruebas y /salud

El pipeline bloquea cambios defectuosos

Despliegue

render.yaml: servicio Docker + base PostgreSQL gestionada

Configuración 12-Factor: todo por variables de entorno, incluidos los parámetros de negocio

Conclusiones y trabajo futuro

- **Lo logrado** Entre los principales logros del proyecto se encuentran la implementación de las reglas del BRD directamente en el dominio, respaldadas con pruebas en valores límite. Asimismo, se logró coordinar los *aggregates* mediante eventos idempotentes y ejecutar con éxito la misma suite de pruebas tanto en SQLite como en un entorno de PostgreSQL real. Finalmente, se garantizó una completa trazabilidad desde el BRD hasta cada regla, componente y prueba.
- **Trabajo futuro** Como parte de las líneas de trabajo futuro para el proyecto, se contempla la incorporación de una autenticación real basada en JWT u OAuth, la implementación del patrón Outbox para la gestión de eventos, y el manejo de migraciones con Alembic junto con el uso de claves foráneas. También se planea integrar un cargo específico por conductor joven y validar los montos directamente con el negocio.